ClearPort — Solution Architecture and System Design

Oct 12, 2026 · Draft v0.1 · Companion to the Customs Clearance BRD v0.1

1. Architecture at a glance

ClearPort is one TypeScript codebase. A React PWA is served from a CDN. The backend is a serverless modular monolith on AWS Lambda, and the data store is a single multi-tenant PostgreSQL database with row-level security. Every component scales to zero, so the prototype runs on roughly USD 5–30 a month (section 13). The same design carries the full BRD scope, through P3, without a rewrite.

Concern	Decision	Why this is the best fit
Client	One installable **PWA**: React 19, TypeScript, Vite, Workbox. It holds three role-based shells: back office, field (PRO, inspector, driver) and customer portal	One codebase for desktop and mobile with no app-store cost; offline field updates; instant updates
Backend style	**Modular monolith on serverless.** There is one Lambda per bounded context (8 functions), not one per endpoint	Fewer cold starts and deployables than micro-functions; module boundaries are kept, so a module can be split out later
Compute	AWS Lambda, Node.js 22, arm64 (Graviton), behind **API Gateway HTTP API**	Pay per request; HTTP API costs about 70% less than REST API; arm64 costs about 20% less than x86
Code structure	**Hexagonal (ports and adapters) + Domain-Driven Design**, with table-driven workflow state machines	Business rules stay testable without AWS; integrations are swappable adapters
Database	**PostgreSQL 16**, serverless: Neon for the prototype, with Aurora Serverless v2 as the in-region production option (ADR-004)	The domain is relational (BL → containers → stages → payments); RLS isolates tenants; both options scale to zero

Concern	Decision	Why this is the best fit
Files	Amazon S3 with presigned upload/download URLs; CloudFront in front of the PWA	Files never pass through Lambda; cheapest storage; no egress through the API
Identity	**Amazon Cognito** (OAuth 2.1 / OIDC, PKCE), with RBAC + attribute-based checks in the API and RLS in the database	Managed, standards-based, free tier covers the prototype; defence in depth
Async	**Transactional outbox → SQS**; EventBridge Scheduler for timers (SLA, free time, DO validity)	Reliable notifications and integrations with no always-on broker
Messaging	Amazon SES (email), WhatsApp Cloud API (direct from Meta), Web Push (VAPID)	Lowest per-message cost; no paid middleman
Infrastructure as code	**AWS CDK (TypeScript)**; GitHub Actions with OIDC deploys to AWS	Reproducible environments; no long-lived cloud keys
Region	One AWS region in the Middle East (me-central-1 UAE or me-south-1 Bahrain), to be confirmed against the client's data-residency rules	Low latency for users; residency compliance

2. Architecture drivers

Four quality attributes rank above everything else: security, cost, performance and usability. Where they conflict, security wins, then cost. Every design choice in this document traces back to one of these drivers or to a BRD requirement ID.

Quality attribute scenarios (measurable, so they can be tested)

ID	Attribute	Scenario	Target	BRD link
QA-1	Security	A user of tenant A crafts an API call with tenant B's BL id	Request returns 404. The database returns zero rows even if the API check is bypassed	NFR-001, NFR-002

ID	Attribute	Scenario	Target	BRD link
QA-2	Security	Any change to a stage, payment or document	An append-only audit event records who, when, before and after; it cannot be edited through the app	NFR-004, ADM-004
QA-3	Cost	Prototype with 1 tenant, 20 users and about 300 BLs a month	Cloud bill under USD 30 a month	—
QA-4	Cost	Idle at night and at weekends	Spend close to zero: no always-on servers	—
QA-5	Performance	Coordinator opens the kanban with 2,000 open BLs	Under 2 s at the 95th percentile (p95) on a warm path	NFR-005
QA-6	Performance	PRO taps "Duty paid" on a mid-range Android phone with weak signal	UI confirms in under 1 s (optimistic, queued offline); syncs when online	NFR-007
QA-7	Usability	A new coordinator creates a BL with 5 containers	Under 10 minutes without training; container numbers pasted in bulk	NFR-009, JOB-002
QA-8	Modifiability	The agency adds a new stage or changes a CCRO gate	Done in admin settings; no deployment	NFR-012, ADM-002
QA-9	Availability	One availability zone fails	No user impact (managed services span zones)	NFR-006

Constraints given

- The client is a Progressive Web App; there are no native apps in this version.
- Backend logic is serverless. Cloud resources are used only where they are necessary.
- Every BRD feature must have a home in the architecture, including P2/P3 ones. The prototype implements a subset (section 15).

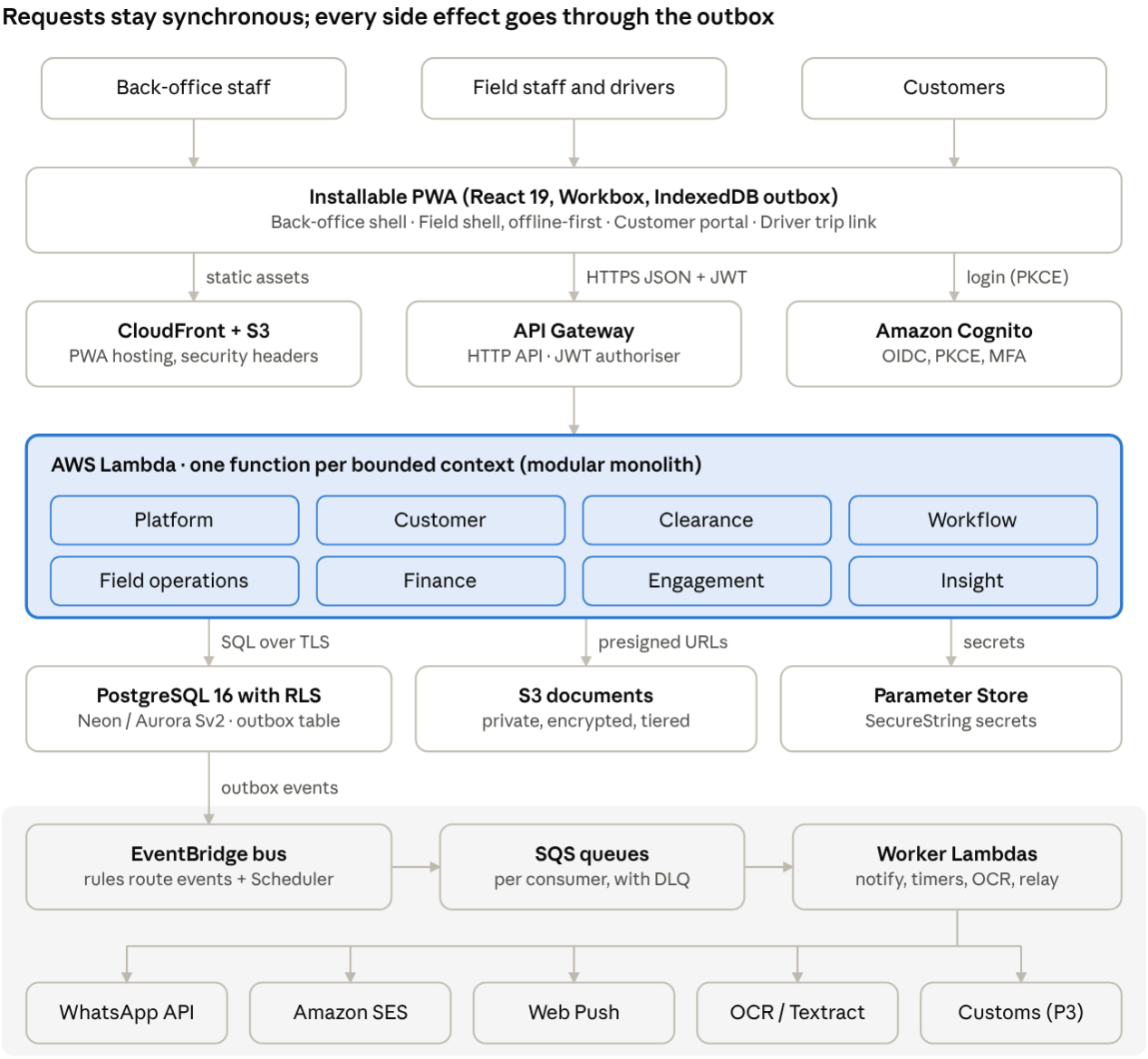
Assumptions

- AWS is the cloud provider. GCP or Azure would work with the same patterns; only the adapters change.

- Volumes per agency are small to medium: up to about 1,000 BLs and 5,000 containers a month. The design scales well beyond this without change.

3. System context and deployment

User requests take a short synchronous path: PWA → API Gateway → one context Lambda → Postgres. Everything else — notifications, timers, document AI and integrations — runs asynchronously from the outbox, so it never slows a user down or gets lost.



System context and deployment · AWS serverless, single region

Files bypass the API: the PWA uploads and downloads directly to S3 with short-lived presigned URLs that the Clearance Lambda issues. API Gateway checks Cognito-issued JWTs before any Lambda runs.

Deployment topology

- One AWS region in the Middle East, two accounts (dev/staging and prod) under AWS Organizations.
- All services are managed and span several availability zones (QA-9).
- No VPC, NAT Gateway or EC2 in the prototype. Lambdas reach Postgres over TLS on the public endpoint. If Aurora is chosen, they use the RDS Data API (ADR-004).
- Separate CloudFront distributions for the app and for the customer portal's custom domain per tenant (P2).

4. Technology stack

The whole stack is TypeScript, from the browser to the database schema. Types, validation schemas and API contracts are shared, so one class of bugs — where the frontend and backend disagree — disappears. Every choice below is mainstream, open-source or pay-per-use, so there is no licence cost.

Layer	Choice	Why	Rejected alternative
UI framework	React 19 + TypeScript 5 (strict) + Vite	Largest talent pool; fast builds; matches the team's existing React work	Angular (heavier bundle); Next.js (needs server rendering, which we don't need for a logged-in app)
PWA	vite-plugin-pwa (Workbox)	Precaching, runtime caching and background sync without hand-writing service workers	Hand-written service worker
Routing and server state	TanStack Router + TanStack Query	Type-safe routes; caching, retries and optimistic updates built in	Redux (more boilerplate for server state)
Offline store	IndexedDB via Dexie	Outbox of pending field actions; cached tasks for offline viewing	localStorage (too small, synchronous)

Layer	Choice	Why	Rejected alternative
UI kit	Tailwind CSS 4 + shadcn/ui (Radix primitives)	Accessible components we own (no lock-in); logical properties give RTL for free	MUI (larger bundle, harder RTL theming)
Forms and validation	React Hook Form + **Zod** (schemas shared with the API)	One schema validates in the browser and on the server	Separate validators per side
i18n	i18next with ICU messages; English and Arabic, RTL	Mature; plural/gender rules for Arabic	—
API runtime	Node.js 22 on Lambda arm64; **Hono** router; AWS Lambda Powertools (logging, metrics, tracing, idempotency, parameters)	Hono is tiny (fast cold start) and standards-based; Powertools gives best-practice ops for free	NestJS (heavy cold start); Java Spring on Lambda (cold starts of seconds without SnapStart; larger memory cost)
API contract	OpenAPI 3.1 generated from Zod (zod-openapi); typed client generated for the PWA	Contract-first, documented, testable	Hand-written docs
Data access	**Drizzle ORM** + SQL migrations (drizzle-kit)	SQL-first, type-safe, small bundle, no query engine binary	Prisma (larger cold start); TypeORM (weaker typing)
Database	PostgreSQL 16 — Neon serverless (prototype) / Aurora Serverless v2 (production in region)	Relational integrity, RLS, JSONB for config, full-text search	DynamoDB (single-table design is a poor fit for ad-hoc ops reports and joins); MongoDB (weaker transactions across documents)
Identity	Amazon Cognito user pools + hosted login (PKCE), with TOTP MFA	Managed security, free tier	Auth0/Clerk (per-user cost grows fast); our own auth (risk)

Layer	Choice	Why	Rejected alternative
Files	Amazon S3 (+ lifecycle to Infrequent Access / Glacier)	Cheapest durable storage	Storing files in the database
Async	SQS (standard and FIFO), EventBridge Scheduler	Pay per use, no broker to run	Kafka/MSK, RabbitMQ (always-on cost)
Email / WhatsApp / push	SES, WhatsApp Cloud API, Web Push (VAPID)	Lowest cost per message	Twilio (adds a margin per message)
Document AI (P2)	Amazon Textract (AnalyzeDocument / Queries), behind a port	Pay per page; strong on tables	A self-hosted model (always-on GPU cost)
Infrastructure as code	AWS CDK v2 (TypeScript)	Same language; AWS-native; reviewable diffs	Terraform (second language); console clicks
CI/CD	GitHub Actions + OIDC role assumption	Free minutes for small teams; no stored keys	Jenkins (a server to run)
Observability	CloudWatch Logs/Metrics (Powertools, structured JSON), X-Ray sampling, Sentry free tier for the PWA	Low cost; end-to-end request IDs	Datadog (cost)
Tests	Vitest, Testcontainers (Postgres), Playwright (end-to-end, including offline mode), k6 (load)	Fast unit tests; real database in integration tests	Mocked databases (miss RLS bugs)

5. Backend design

The backend is split into 8 bounded contexts. Each one is deployed as its own Lambda behind a route prefix, and each owns its own Postgres schema. Inside each context, a hexagonal structure keeps business rules free of AWS, HTTP and SQL. Four small worker Lambdas handle asynchronous work.

Bounded contexts

#	Context (Lambda)	Owns	BRD requirements	Route prefix
1	Platform	Tenants, users, roles, permissions, master data (ports, lines, fee types, document types), stage configuration	ADM-001…006	`/v1/admin`
2	Customer	Customers, contacts, credit terms, tariffs	CUS-001…005	`/v1/customers`
3	Clearance	BL aggregate, containers, documents (metadata), comments	JOB-001…008, DOC-001…004	`/v1/jobs`
4	Workflow	Stage instances, gates, status transitions, SLA timers, tasks	BAY, DO, INS, CCR, OPS-004/005	`/v1/jobs/{id}/stages`, `/v1/tasks`
5	Field operations	Drivers, vehicles, transporters, trips, physical inspection, release, delivery, POD	TRK, PHY, REL, DEL	`/v1/trips`, `/v1/containers/{id}/…`
6	Finance	Disbursements, PRO floats, invoices, receivables, quotations	FIN-001…008	`/v1/finance`
7	Engagement	Notifications, templates, customer approvals, customer portal API	NTF, CXP, BAY-005	`/v1/portal`, `/v1/notifications`
8	Insight	Read models for kanban, dashboards and reports	OPS-001…003/006, reports	`/v1/insight`

#	Context (Lambda)	Owns	BRD requirements	Route prefix
W	Workers	Outbox relay, notification sender, scheduled-timer handler, document AI processor	—	SQS / Scheduler triggers

Inside a context (hexagonal layout)

```
packages/
  domain-kernel/         shared value objects: TenantId, Money, ContainerNo
(ISO 6346), Result<T,E>
  contracts/             Zod schemas + OpenAPI — shared by API and PWA
  modules/clearance/
    domain/              aggregates, entities, value objects, domain events,
policies (pure TS, no I/O)
    application/         commands, queries, handlers; ports (interfaces) e.g.
JobRepository, FileStore, Clock
    adapters/
      http/              Hono routes → validate with Zod → call a
command/query
      persistence/       Drizzle repositories implementing ports
      messaging/         outbox writer, event mappers
    public/              the only API other modules may import (facade +
published events)
  apps/api-clearance/    Lambda entry: composition root wires adapters to
ports
```

Dependencies point inwards only: adapters → application → domain. A lint rule (dependency-cruiser) fails the build if a module imports another module's internals, or if the domain layer imports any I/O library. Because all modules share one repository and one database, a context reads another context's data through that context's `public/` facade in-process. It never queries the other context's tables directly. This keeps the system a modular monolith that can later be split into services without rewriting.

Design patterns and where they apply

Pattern	Where	Why it is the best fit
Hexagonal architecture (ports and adapters)	Every context	Business rules unit-tested without AWS; swap Neon for Aurora, or SES for another provider, by changing one adapter
Aggregate + Repository + Unit of Work	BL (root) with containers; Trip; Invoice	One transaction per command protects invariants (e.g. container uniqueness, totals)
CQRS (lightweight)	Commands write normalised tables; Insight reads from projection tables and materialised views	Kanban and dashboards stay fast without complex joins on the write model
Domain events + Transactional Outbox	Every state change writes an event row in the same transaction; a relay publishes to SQS	No lost notifications or integrations; no two-phase commit
Table-driven State Machine (State pattern)	Stage lifecycle in the workflow engine	Stages, statuses and transitions are data, so agencies configure them (QA-8)
Specification pattern	Gate rules, e.g. "CCRO requires DO received AND inspection completed AND PRO assigned"	Composable, testable rules that explain which condition fails
Strategy pattern	Fee calculation (per BL, per container, slab), notification channel (WhatsApp, email, SMS, push), document extraction provider	New tariff types and channels without touching callers
Adapter + Anti-corruption layer	Customs, port, shipping-line and accounting integrations (P3)	External models never leak into our domain
Policy-based authorisation (RBAC + ABAC)	A single `can(user, action, resource)` policy layer per context	Stage-level permissions; branch and customer scoping

Pattern	Where	Why it is the best fit
Idempotency key	All POSTs from the PWA and every queue consumer	Offline retries and SQS redelivery never double-post a payment
Optimistic concurrency (version column)	BL, stage, trip updates	Two coordinators editing the same BL get a clear conflict instead of a silent overwrite
Factory	Creating a job from intake, email-in or document AI	One place builds a valid BL with its container list and initial stages
Result type instead of exceptions for domain errors	Domain and application layers	Explicit, typed error handling mapped to RFC 9457 problem responses
Retry with backoff + dead-letter queue + circuit breaker	Outbound adapters (WhatsApp, SES, Textract)	Resilience without blocking user requests

6. Workflow engine

The 9-stage clearance flow is stored as versioned configuration, not code. A small, generic engine reads the configuration, checks gates with specifications, applies the transition, and emits a domain event. Agencies can add stages or change gates in admin settings (ADM-002). Running jobs keep the configuration version they started with.

Stage definition (stored as JSONB, versioned per tenant)

```
{
  "key": "CCRO",
  "level": "BL",
  "label": { "en": "CCRO", "ar": "..." },
  "statuses": ["NOT_STARTED", "APPLIED", "ISSUED"],
  "initial": "NOT_STARTED",
  "transitions": [
    { "from": "NOT_STARTED", "to": "APPLIED", "permission":
"stage.ccro.update",
      "guard": { "all": [
        { "stage": "DO", "status": "DO_RECEIVED" },
        { "stage": "INSPECTION", "statusIn": ["CLEARED", "COMPLETED"] },
        { "assigneeRole": "PRO" } ] },
      "requiredFields": [] },
    { "from": "APPLIED", "to": "ISSUED", "permission": "stage.ccro.update",
      "requiredFields": ["referenceNo", "issuedOn"], "requiresDocument":
"CCRO_COPY" }
  ],
  "sla": { "targetHours": 24, "warnAtPercent": 75 },
  "onEnter": { "ISSUED": ["spawn:CONTAINER_STAGES",
"notify:CUSTOMER_CLEARED"] }
}
```

How one transition is processed (for example, a PRO sets Bayan to "Duties paid")

1. The API validates the request body with Zod and checks the idempotency key.
2. The policy layer checks the permission (`stage.bayan.update`) and the user's scope (branch, assignment).
3. The handler loads the stage instance and the configuration version it was created with.
4. The engine finds the transition and evaluates its guard as a composed Specification. If it fails, the API returns a 409 problem response that lists each unmet condition. The UI shows these as "Waiting for: DO received".
5. Required fields and documents are checked; any payments in the same request are attached to the stage.
6. In one database transaction, the engine updates the stage (with an optimistic version check), appends to `stage_event` (the audit trail), and writes `StageStatusChanged` to the outbox.
7. After commit, the outbox relay publishes to SQS. Consumers then update the Insight projections, send notifications, re-arm SLA timers, and run `onEnter` actions. For example, when the CCRO is issued, the engine creates the container-level stages for every container.

Derived states

- A container's current stage is the latest open container-level stage.
- The BL's overall status is computed from its stages and containers (for example "Delivering — 3/5 delivered") and stored in the projection.
- BL closure (DEL-004) is a guarded transition: every container delivered, and every disbursement reconciled.

Timers — SLA breaches, free-time expiry (T-3, T-1, T-0) and DO validity are one-off EventBridge Scheduler schedules, created and cancelled by event consumers. Nothing polls the database on a cron.

7. Data architecture

All tenants share one PostgreSQL database. Every business table carries `tenant_id`, and Postgres row-level security (RLS) enforces isolation in the database itself. Each bounded context owns a schema. Files live in S3, and the database stores only their metadata.

Tenant isolation in three layers

1. The API takes the tenant from the verified JWT claim. It never takes it from the URL or the request body.
2. Every request runs in a transaction that starts with `SET LOCAL app.tenant_id = '<uuid>'`.
3. Every table has `ENABLE ROW LEVEL SECURITY` + `FORCE ROW LEVEL SECURITY`, with the policy `tenant_id = current_setting('app.tenant_id')::uuid`. The application connects as a non-owner role without `BYPASSRLS`. Migrations run as a separate owner role from CI only.

An automated test suite tries cross-tenant reads and writes on every table and must get zero rows (QA-1).

Data conventions

- Primary keys are UUIDv7: time-ordered, so inserts stay index-friendly, and safe to generate offline in the PWA.
- Money is `numeric(14,3)` plus an ISO currency code. Three decimals cover KWD, BHD and OMR; no floating point anywhere.
- Timestamps are `timestamptz`, stored in UTC and shown in the tenant's time zone.
- Every aggregate root has a `version int` for optimistic locking, plus `created_at/by` and `updated_at/by`.
- Records are never hard-deleted: `archived_at`. Audit and event tables allow insert only (`REVOKE UPDATE, DELETE`).

Core tables by schema

Schema	Tables	Key constraints and indexes
platform	tenant, app_user, role, role_permission, user_role, branch, port, terminal, shipping_line, fee_type, document_type, stage_definition (JSONB, versioned)	unique(tenant_id, email); stage_definition unique(tenant_id, key, version)
customer	customer, customer_contact, tariff, tariff_line	unique(tenant_id, customs_code)
clearance	job (BL), container, document, document_version, comment	unique(tenant_id, bl_no); partial unique index on container(tenant_id, container_no) WHERE job is open (JOB-003); trigram index on bl_no and container_no for search
workflow	stage_instance, stage_event, task, gate_snapshot	index(tenant_id, stage_key, status); index(tenant_id, assignee_id, status) for "My tasks"
fieldops	driver, vehicle, transporter, trip, trip_container, inspection_result, delivery, pod	index(tenant_id, driver_id, status); expiry dates on licences and registrations
finance	disbursement, pro_float, float_settlement, invoice, invoice_line, payment_receipt, quotation	disbursement index(job_id); invoice unique(tenant_id, invoice_no) with a gap-free per-tenant sequence
engagement	notification, notification_template, customer_approval, portal_access	index(tenant_id, status, created_at)
insight	job_board (projection), container_board (projection), stage_duration_fact; materialised views for reports	Rebuilt from events, so they can be dropped and replayed
shared	outbox, idempotency_key, audit_event, inbox (for deduplicating consumers)	outbox index(published_at) WHERE published_at IS NULL

Files

- The S3 key is `tenant/{tenantId}/job/{jobId}/{documentId}/v{n}`. The bucket blocks all public access and uses server-side encryption.

- Uploads use a presigned POST with a content-type allow-list (PDF, JPEG, PNG, HEIC), a 20 MB size limit and a 5-minute expiry. The PWA compresses camera photos before upload.
- Downloads use presigned GET URLs that expire after 5 minutes, issued only after a permission check.
- A lifecycle rule moves files to Infrequent Access after 90 days and to Glacier Instant Retrieval after 1 year. Retention follows DOC-004.

Backup and recovery — Point-in-time restore: Neon 7 days on the paid plan; Aurora 7–35 days. Production adds a nightly logical dump to a separate, versioned, object-locked S3 bucket. The restore drill is run quarterly.

8. API design

The API is a versioned REST/JSON API, contract-first with OpenAPI 3.1. The contract is generated from the same Zod schemas the PWA uses, and CI fails on any breaking change to a published version.

Conventions

Topic	Standard
Versioning	URI major version `/v1`; additive changes only within a version; deprecation via `Sunset` header (RFC 8594)
Resources	Plural nouns; actions that are state transitions are explicit sub-resources, e.g. `POST /v1/jobs/{id}/stages/{key}/transitions`
Errors	RFC 9457 Problem Details: `type`, `title`, `status`, `detail`, `errors[]`; gate failures list the unmet conditions
Pagination	Cursor-based (`?cursor=&limit=`, max 100) on all lists; no offset paging on large tables
Filtering and sorting	`?status=OPEN&customerId=...&sort=-eta`; allow-listed fields only
Idempotency	`Idempotency-Key` header required on every POST; results stored 24 h (Powertools idempotency)
Concurrency	`ETag` / `If-Match` carrying the aggregate version; 412 on conflict

Topic	Standard
Time and money	ISO 8601 UTC timestamps; money as `{ "amount": "1250.000", "currency": "AED" }` (string amounts, no float)
Bulk operations	`POST /v1/jobs/{id}/containers:bulk` (up to 200); partial-success report
Correlation	`X-Request-Id` generated at the edge, propagated to logs, events and outbound calls
Rate limits	API Gateway throttling per stage + per-tenant usage plan in the Platform context

Endpoint catalogue (core)

Method and path	Purpose	BRD
`POST /v1/jobs`	Create a BL job with containers and initial stages	JOB-001…005
`GET /v1/jobs?stage=&customerId=&q=`	Search and list jobs	OPS-006
`GET /v1/jobs/{id}`	BL detail with containers, stages and totals	OPS-003
`GET /v1/jobs/{id}/timeline`	Ordered events, payments, documents, comments	OPS-003
`POST /v1/jobs/{id}/documents:presign`	Get an upload URL; then `POST /v1/jobs/{id}/documents` to register it	DOC-001…003
`POST /v1/jobs/{id}/stages/{key}/transitions`	Move a stage (Bayan, DO, Inspection, CCRO…); 409 lists unmet gates	BAY, DO, INS, CCR
`PUT /v1/jobs/{id}/stages/{key}/assignee`	Assign a PRO or agent	BAY-001, DO-002
`POST /v1/stages/{stageId}/disbursements`	Record an amount paid with a receipt	BAY-003, FIN-001
`POST /v1/trips`	Assign truck, driver, inspection agent and delivery to one or more containers	TRK-001/002

Method and path	Purpose	BRD
`POST /v1/trips/{id}/events`	Driver updates: accepted, at port, gated out, delivered (+ POD)	TRK-004, REL-001, DEL-001/002
`POST /v1/containers/{id}/inspection`	Physical inspection result with photos	PHY-001…003
`POST /v1/jobs/{id}/close`	Guarded closure; 409 lists what blocks it	DEL-004
`GET /v1/tasks?mine=true`	My tasks across stages and trips	OPS-005
`GET /v1/insight/board`	Kanban projection by stage	OPS-001/002
`POST /v1/finance/invoices`	Invoice from tariff + billable disbursements	FIN-003
`GET /v1/portal/jobs`	Customer portal: own BLs only, customer-visible fields	CXP-001
`POST /v1/portal/approvals/{token}`	One-tap customer approval of duty and charges	BAY-005
`POST /v1/webhooks/whatsapp`	Delivery receipts and replies (signature verified)	NTF-002

9. Frontend PWA

One PWA serves every role. After login it loads only the shell for that role. Field users get an offline-first, one-thumb interface; the back office gets dense, keyboard-friendly screens; and customers get a light, read-mostly portal.

App shells

Shell	Users	Key screens	Offline
Back office	Coordinator, manager, accounts, admin	Kanban by stage; BL detail with a stage rail and timeline; container board; tasks; disbursement ledger; invoices; reports; settings (stage configuration, master data, users)	Read-only cache of recently opened BLs
Field	PRO, inspection agent, driver	Today's tasks; stage update with a camera receipt; trip card with a big status button; inspection with photos; POD signature pad	**Full**: actions are queued and synced later
Customer portal	Importer contacts	My shipments; container status; documents, invoices and POD; approve charges	Read-only cache
Driver trip link	Third-party drivers without an account	A single trip page opened from WhatsApp/SMS with a signed, expiring token	Queued status updates

Offline-first design (field shell)

- The app shell and static assets are precached by Workbox, so the app opens with no signal.
- Each user action becomes a command in an IndexedDB outbox with a client-generated UUIDv7 and an `Idempotency-Key`. The UI updates optimistically and shows a "pending sync" badge.
- The sync runs on reconnect, on app focus, and through the Background Sync API where the browser supports it. iOS Safari lacks Background Sync, so the app falls back to syncing on focus and on a timer.
- Photos are compressed on the device (max 1600 px, WebP/JPEG at about 75% quality) before they are queued, typically 200–400 KB each.
- On a conflict (412 or 409), the server state wins. The user sees what changed and can reapply their update with one tap. Payments are never silently dropped.

Usability standards

- WCAG 2.2 AA: contrast, focus order, labels, and touch targets of at least 44 px.
- Full Arabic RTL using CSS logical properties. Dates and numbers are formatted with `Intl`, and Western or Arabic-Indic digits are a user setting.

- Field flows take 3 taps or fewer per update (NFR-009). Bulk paste and the device camera help capture container numbers.
- Long-running states such as "Waiting for: DO received" are always shown in words, never only as a colour.
- Installable with a web app manifest. Web Push notifications work on Android and desktop, and on iOS 16.4+ once the app is installed to the home screen.

Performance budget (enforced in CI with Lighthouse CI and bundle-size checks)

Metric	Budget
Initial JS per shell (gzip)	⩽ 170 KB
Largest Contentful Paint (mid-range Android, 4G)	⩽ 2.5 s
Interaction to Next Paint (INP)	⩽ 200 ms
Lighthouse PWA / Accessibility / Best Practices	⩾ 90

The budgets rely on route-level code splitting, lazy-loaded heavy components (signature pad, charts, PDF preview), virtualised long lists (TanStack Virtual) and HTTP caching of immutable hashed assets for 1 year.

10. Security architecture

Security is layered, so no single failure exposes data. The baseline is OWASP ASVS Level 2, the OWASP Top 10 and the OWASP API Security Top 10, plus least-privilege IAM. A penetration test is required before the first paying tenant beyond the pilot.

Identity and sessions

- Staff sign in through Cognito hosted login using the Authorization Code flow with PKCE. MFA (TOTP) is mandatory for the owner, accounts and admin roles (NFR-002), and optional for others.
- The API uses the token-handler pattern. A small `/auth` Lambda exchanges the code and keeps the refresh token in an `HttpOnly; Secure; SameSite=Strict` cookie scoped to `/auth`. The PWA holds only a 15-minute access token, in memory, never in localStorage. This removes token theft through XSS from storage.
- Customers sign in to the portal with an email one-time code, using a separate Cognito app client and a `customer` group.
- Drivers without an account open a trip link that holds a random 256-bit token. Only its hash is stored. It is scoped to one trip, expires after 48 h and can be revoked.

Controls by layer

Layer	Controls
Edge (CloudFront)	TLS 1.2+ only; HSTS; strict Content Security Policy (no inline script, `frame-ancestors 'none'`); Referrer-Policy; Permissions-Policy (camera only where needed). AWS WAF managed rules + rate rules in production (about USD 10 a month; skipped in the prototype)
API Gateway	JWT authoriser (Cognito issuer and audience); route throttling; payload size limits; CORS restricted to the app origin
Application	Zod validation on every input (deny unknown fields); policy layer `can(user, action, resource)` checks role, branch, assignment and customer scope; idempotency; RFC 9457 errors that never leak stack traces; no PII in logs (Powertools log masking)
Data	RLS per tenant (section 7); non-owner application role; parameterised SQL only (Drizzle); encryption at rest (Neon / Aurora default + S3 SSE); TLS to the database
Files	Presigned URLs with 5-minute expiry; content-type allow-list + magic-byte check in a worker; private bucket; malware scanning with GuardDuty for S3 from P2
Secrets	SSM Parameter Store SecureString (free tier) for DB URL, WhatsApp token, VAPID keys; read at cold start through the Powertools parameters cache; never in code or environment files
Cloud account	One Lambda execution role per function, least privilege; separate AWS accounts for dev and prod (AWS Organizations); CloudTrail on; root account with MFA and no keys; budget alarms
Supply chain	Locked dependencies; Dependabot; `npm audit` and CodeQL/Semgrep in CI; SBOM (CycloneDX) per release; signed commits on main
Audit	Append-only `audit_event` for every write, with actor, IP, user agent, before/after diff; login and permission-change events; retention per DOC-004

Threat model highlights (STRIDE)

Threat	Example	Mitigation
Spoofing	Stolen PRO phone used to post payments	Short-lived tokens; remote session revoke; MFA for finance roles; device list per user
Tampering	A user edits a paid amount after reconciliation	Reconciled disbursements are immutable; corrections are reversal entries; full audit
Repudiation	"I never marked that container delivered"	Signed POD with geotag and timestamp; audit event with actor and device
Information disclosure	Customer A guesses customer B's BL id	UUIDs plus scope checks plus RLS; portal returns 404, not 403
Denial of service	Bot floods the trip-link endpoint	API throttling, WAF rate rules, token lookup by hash with constant-time compare
Elevation of privilege	Coordinator calls an admin endpoint	Permission checks on every route, generated from one policy map, plus tests per role

Compliance — Data is hosted in-region where the residency rules require it (ADR-004). The design supports the applicable data-protection law (e.g. UAE PDPL): data-subject export and erasure of personal fields, consent for WhatsApp messaging (opt-in stored per contact), and a breach-response runbook.

11. Performance and scalability

The two risks of a serverless design are cold starts and running out of database connections. Both are handled by design, not by paying for always-on capacity.

Concern	Approach	Target
Lambda cold start	esbuild bundles per function, tree-shaken, under 1 MB; arm64; 1024 MB memory (more CPU, a shorter bill); clients and configuration created outside the handler; no heavy frameworks	Cold start p95 < 600 ms; warm p95 < 150 ms server time
Few large functions	8 context Lambdas instead of about 80 per-endpoint ones, so each stays warm under normal office traffic	Cold-start rate < 2% of requests in business hours
Database connections	Prototype: Neon serverless driver plus Neon's built-in PgBouncer pooler. Production on Aurora: RDS Proxy, or the RDS Data API. Reserved concurrency per Lambda caps total connections	No connection exhaustion at 10× pilot load
Database auto-suspend	Neon suspends after 5 minutes idle; the first query after that adds about 0.5 s. Production keeps a minimum capacity during business hours (07:00–20:00), using a schedule	Acceptable for prototype; invisible in production
Read-heavy screens	Kanban, container board and dashboards read Insight projection tables (CQRS), each one indexed query	Board query < 50 ms for 2,000 open BLs
Search	Postgres trigram indexes on BL, container, Bayan, DO numbers; full-text search on customer names	< 150 ms
Client caching	TanStack Query stale-while-revalidate; ETag + 304 Not Modified; hashed static assets cached for 1 year at CloudFront	Repeat views feel instant

Concern	Approach	Target
Payload size	Cursor pagination; sparse field sets on lists; gzip/brotli response compression in the Lambda	List responses < 50 KB
Heavy work off the request path	PDF generation, OCR, notifications, projections run in workers via SQS	User-facing p95 unaffected
Proof	k6 load test in CI on staging: 50 concurrent users, mixed scenario; regression budget enforced	Matches QA-5 and QA-6

Scaling headroom — Lambda and SQS scale automatically. Postgres is the first limit, and only at roughly 100× pilot volume. The remedies, in order: add read replicas for Insight, move the projections to their own database, then split the busiest context into its own service and database. The module boundaries already allow this (section 5).

12. Async processing and integrations

Every state change produces a domain event through the transactional outbox. The events flow over one EventBridge bus to SQS queues, one queue per consumer. All consumers are idempotent: an inbox table removes duplicates. Every queue has a dead-letter queue and a CloudWatch alarm.

Event flow

1. A command commits its business change and its event row in the same transaction.
2. Fast path: right after commit, the API Lambda publishes the new outbox rows to the EventBridge bus and marks them published.
3. Safety net: an outbox relay Lambda runs every minute through EventBridge Scheduler. It republishes any rows left unpublished, for example after a crash between commit and publish.
4. EventBridge rules route events to the consumer queues:
   - `projection-q` : Insight read models
   - `notify-q` : notifications
   - `timer-q` : creates or cancels SLA, free-time and DO-validity schedules
   - `docai-q` : document extraction (P2)
   - `integration-q` : external systems (P3)

5. Each consumer Lambda processes batches with partial-batch failure reporting. After 5 failed attempts, a message goes to its dead-letter queue (DLQ).

Key domain events — `JobCreated`, `ContainerAdded`, `StageStatusChanged`, `GateSatisfied`, `DisbursementRecorded`, `CustomerApprovalRequested` / `Granted`, `CcroIssued`, `TripAssigned`, `ContainerInspected`, `ContainerReleased`, `ContainerDelivered`, `JobClosed`, `SlaBreached`, `FreeTimeExpiring`.

Integrations

Integration	Pattern	Phase	Notes
WhatsApp Cloud API (Meta)	Outbound templates via `notify-q`; inbound webhook with signature verification	P1	Template approval needed; opt-in stored per contact; first 1,000 service conversations a month free *[verify current Meta pricing]*
Amazon SES	Outbound email via `notify-q`; inbound email-in via an SES receipt rule → S3 → worker	P1 / P2	Verify the sending domain with DKIM, SPF and DMARC
Web Push	VAPID; subscriptions stored per device	P1	No third-party service needed
SMS	Amazon SNS SMS or a local aggregator, behind the channel Strategy	P1 fallback	Cost per SMS varies by country
Amazon Textract	Asynchronous AnalyzeDocument with Queries, from `docai-q`; results mapped by an anti-corruption adapter	P2	Check whether Textract is offered in the chosen ME region; else process in the nearest region with a residency waiver *[Confirm]*
Accounting	CSV/Excel export first; then connectors (Zoho Books, QuickBooks) behind an `AccountingPort`	P2	OAuth tokens stored in Parameter Store
Payment gateway	Hosted payment page plus signed webhook	P3	Local acquirer

Integration	Pattern	Phase	Notes
Customs (Bayan), port community, shipping-line systems	One anti-corruption adapter per system; polling via Scheduler or webhooks where offered	P3	Depends on API access and agent credentials
Vessel tracking (AIS)	Scheduled pull, which updates ETA and triggers a `JobEtaChanged` event	P3	Data provider subscription

13. Cost model

The prototype costs about USD 5–30 a month in cloud fees. Five paying agencies cost about USD 100–180 a month, before pass-through messaging and document AI fees, which are billed on to customers. *Figures are approximate, based on public AWS, Neon and Meta list prices known to us; Middle East regions are typically 10–20% above us-east-1. Validate them in the AWS Pricing Calculator before budget sign-off.*

Monthly estimate (USD)

Volumes assumed:

- Prototype: 1 tenant, 20 users, about 300 BLs and 0.3 M API calls a month.
- Scale: 5 tenants, 150 users, about 2,500 BLs and 5 M API calls a month.

Item	Prototype	5 agencies	Note
Lambda (arm64)	0	3–6	Always-free tier: 1 M requests and 400k GB-seconds a month
API Gateway HTTP API	0.5	5	USD 1.00 per million requests
PostgreSQL	0–20 (Neon free or entry plan)	45–90 (Aurora Serverless v2 in region, paused overnight)	The largest fixed line; see ADR-004
S3 + CloudFront	0.5	3–5	CloudFront always-free tier covers 1 TB a month
Cognito	0	0	Lite tier free up to 10,000 monthly active users

Item	Prototype	5 agencies	Note
SQS, EventBridge, Scheduler	0	1–2	Mostly within free tiers
SES email + Web Push	< 1	1–3	USD 0.10 per 1,000 emails; push is free
CloudWatch, CloudTrail, Parameter Store	0–3	10–20	Log retention 30 days; Parameter Store standard tier is free
AWS WAF	—	10–15	Production only
Domain + Route 53	1.5	1.5	
Platform subtotal	**about 5–30**	**about 100–180**	
WhatsApp template messages	10–40	150–400	Pass-through: priced per message by Meta and varies by country *[verify]*
Document AI (P2)	—	20–200	Textract AnalyzeDocument is about USD 15 per 1,000 pages. The cheaper route is OCR text (about USD 1.5 per 1,000 pages) plus a small LLM for field extraction, behind the same port (ADR-007)

Cost guardrails built into the design

- There are no always-on compute resources. There is no NAT Gateway (USD 30+ a month each): Lambdas run outside a VPC and reach Neon or the Aurora Data API over TLS.
- No RDS Proxy or provisioned concurrency in the prototype. Both are turned on only when metrics show they are needed.
- Logs are kept for 30 days, then archived to S3 if required. Debug logging is sampled at 1%.
- S3 lifecycle tiers for documents. Photos are compressed on the device before upload.
- AWS Budgets alarms at 50%, 80% and 100% of the monthly budget, plus anomaly detection, per account.

- Every resource is tagged by environment and cost centre. Usage is metered per tenant (BLs, containers, messages, OCR pages), so the pricing tiers in the BRD can be checked against real cost.

14. Engineering practices

Every change reaches production through one automated pipeline, with tests, security scans and performance budgets as gates. Nothing is deployed by hand.

Repository — One pnpm + Turborepo monorepo: `apps/web` (PWA), `apps/api-*` (one per context), `apps/workers`, `packages/*` (domain kernel, contracts, modules, UI kit), `infra/` (CDK). Builds are cached per package, so CI stays fast.

Branching and review — Trunk-based development with short-lived branches. Pull requests need one approving review and green CI. Commits follow Conventional Commits, and semantic versioning and changelogs are generated from them. Architecture decisions are recorded as ADRs in `docs/adr`.

Environments

Environment	Purpose	Database	Deploy
Preview (per pull request)	Review a change in isolation	Neon branch, copied from staging in seconds, at no extra cost	Automatic; torn down on merge
Staging	Integration, end-to-end and load tests; stakeholder demos	Neon / Aurora staging	Automatic on merge to main
Production	Live tenants	Aurora Serverless v2 (in region) or Neon	Manual approval; canary through Lambda aliases (10% for 10 minutes, auto-rollback on alarm)

CI/CD pipeline (GitHub Actions)

1. Install, then typecheck, lint (ESLint, Prettier, dependency-cruiser boundaries) and unit tests (Vitest).
2. Integration tests against Postgres in Testcontainers, including the RLS cross-tenant suite and a migration up/down check.
3. Security: CodeQL/Semgrep, `npm audit`, secret scanning, SBOM.

4. Build the bundles; Lighthouse CI and bundle-size budgets; OpenAPI breaking-change diff.
5. `cdk diff`, then deploy to preview or staging through an OIDC role.
6. Playwright end-to-end tests (desktop and mobile viewports, including an offline scenario), then a k6 smoke load test.
7. Promote to production after approval, then run a post-deploy smoke test.

Testing strategy

- Unit tests cover the domain and application layers, including the workflow engine, gates and money maths. Coverage must be at least 80%, and 100% on the gate specifications.
- Integration tests cover repositories, RLS, the outbox and idempotency.
- Contract tests check the API against OpenAPI.
- End-to-end tests follow one full BL from intake to closure.

Observability — Structured JSON logs with request, tenant and user IDs. Custom business metrics (BLs created, stage durations, SLA breaches). X-Ray tracing at 5% sampling. CloudWatch alarms on 5xx rate, p95 latency, DLQ depth and budget. Sentry on the PWA. One dashboard per environment.

Definition of done — The code is reviewed and has tests. Its API is documented and accessibility-checked. It is translated (English and Arabic), shipped behind a feature flag where it is risky, and observable in the dashboard.

15. Prototype delivery plan

The stakeholder prototype runs the full P1 flow end to end on real infrastructure. It also shows every P2/P3 feature in working form, either real or behind a stub adapter, so stakeholders see the whole BRD. The estimate is 11 weeks with a team of 4.5 *[Confirm team]*: an architect/lead, 2 full-stack engineers, a part-time QA engineer and a part-time UX designer.

What the prototype includes

BRD area	In the prototype	How
P1 core flow (JOB, BAY, DO, INS, CCR, TRK, PHY, REL, DEL, OPS, DOC, ADM)	Fully working	Real modules, real data
Disbursements and PRO floats (FIN-001/002)	Fully working	Real

BRD area	In the prototype	How
WhatsApp / email notifications	Working	WhatsApp Cloud API test number; SES sandbox
Free-time guard, invoicing, customer portal (P2)	Working, with simplified rules	Real modules, reduced scope
Document AI intake (P2)	Demonstrated	Real port, using an OCR adapter on sample BLs
Customs, port and line integrations; marketplace; prediction (P3)	Clickable demo	Stub adapters with recorded responses; shows the integration seam

Increments

1. **Sprint 0 (1 week)** — Monorepo, CDK stacks, CI/CD, Cognito, Neon, the RLS baseline, the design system and the app shells.
2. **Sprint 1 (2 weeks)** — Customers, BL intake with containers, documents, the workflow engine, and the Bayan and DO stages with disbursements.
3. **Sprint 2 (2 weeks)** — Inspection, the CCRO gate, the kanban and BL timeline, and "My tasks". This ends with **Demo 1**: the back-office flow up to CCRO.
4. **Sprint 3 (2 weeks)** — The field shell, offline: PRO updates, truck assignment, physical inspection and delivery with POD, the driver trip link, and BL closure.
5. **Sprint 4 (2 weeks)** — Notifications (WhatsApp, email, push), customer portal and approvals, free-time guard, invoicing and reports.
6. **Sprint 5 (2 weeks)** — Document AI demo, P3 stubs, Arabic/RTL polish, performance and security hardening, load test, and seeding realistic demo data. This ends with **Demo 2**, the stakeholder showcase.

16. Decisions, risks and open items

Eight architecture decisions are proposed for sign-off. The most consequential is ADR-004, the production database: it depends on whether the client's data must stay in-country.

Architecture decision records

ADR	Decision	Main alternative rejected	Status
ADR-001	PWA, not native apps	React Native (store fees, two release trains)	Proposed

ADR	Decision	Main alternative rejected	Status
ADR-002	Modular monolith on Lambda, one function per bounded context	Microservices / Lambda per endpoint (cold starts, operational overhead)	Proposed
ADR-003	TypeScript end to end (Hono, Drizzle, Zod)	Java Spring Boot on Lambda (cold start, memory cost)	Proposed
ADR-004	PostgreSQL with RLS; Neon for prototype, Aurora Serverless v2 in region for production if residency requires it	DynamoDB (poor fit for relational reporting)	Proposed; **needs residency answer**
ADR-005	Cognito with the token-handler pattern; no tokens in browser storage	Tokens in localStorage (XSS exposure)	Proposed
ADR-006	Transactional outbox + EventBridge + SQS for all side effects	Direct calls from request handlers (lost messages)	Proposed
ADR-007	Document AI behind a port; start with OCR + small LLM, keep Textract Queries as an option	Hard-wiring one vendor	Proposed
ADR-008	Workflow stages as versioned configuration	Hard-coded stage logic	Proposed

Risks

Risk	Impact	Mitigation
iOS PWA limits (no Background Sync; push only after the app is installed to the home screen)	Field users on iPhone sync only when the app is open	Sync on focus and on a timer; in-app install prompt; offline banner; measure the share of iOS users in the pilot
Serverless Postgres resumes slowly after idle	First request of the morning is slow	Keep minimum capacity in business hours in production
The Middle East region lacks a needed service (e.g. Textract)	Feature delay or residency conflict	Ports and adapters allow another provider or region; check before P2

Risk	Impact	Mitigation
Team new to the serverless and DDD patterns	Slower early sprints	Sprint 0 reference module (Customer) used as the template; pairing; ADRs
Cost growth from messaging and OCR	Margin erosion	Per-tenant metering; quotas by tier; pass-through billing

Open decisions for stakeholders

- [ ] Country and data-residency requirement, which decides ADR-004 and the region.
- [ ] Confirm AWS as the cloud provider, and who owns the AWS organisation and billing.
- [ ] Team composition and start date for Sprint 0.
- [ ] Product name and domain ("ClearPort" is a working name).
- [ ] WhatsApp Business account owner: one platform number, or one per agency.

Next step: review this design with the tech lead and the client sponsor, then start Sprint 0.